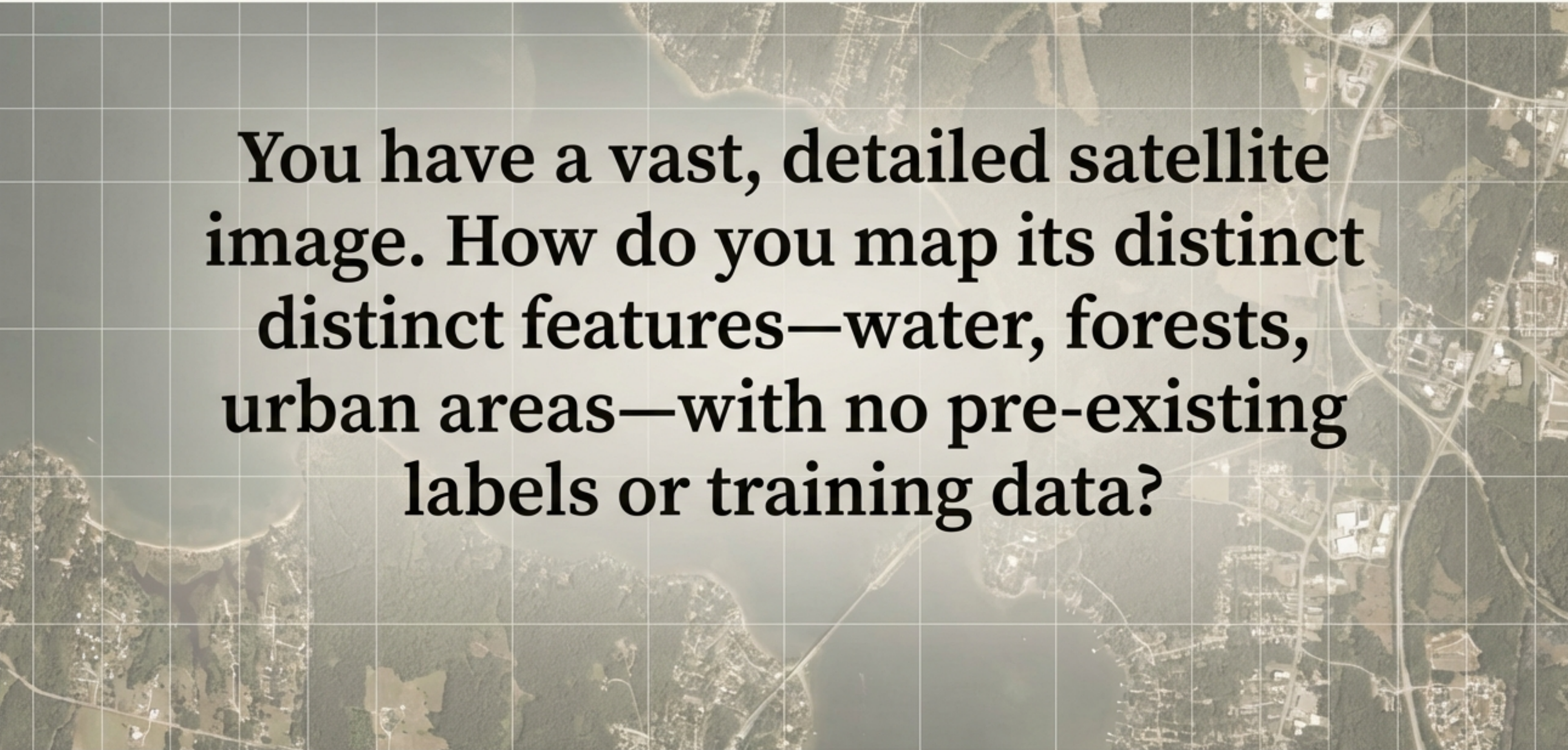




Decoding the Digital Landscape

A Practical Guide to Unsupervised
Classification in Google Earth Engine

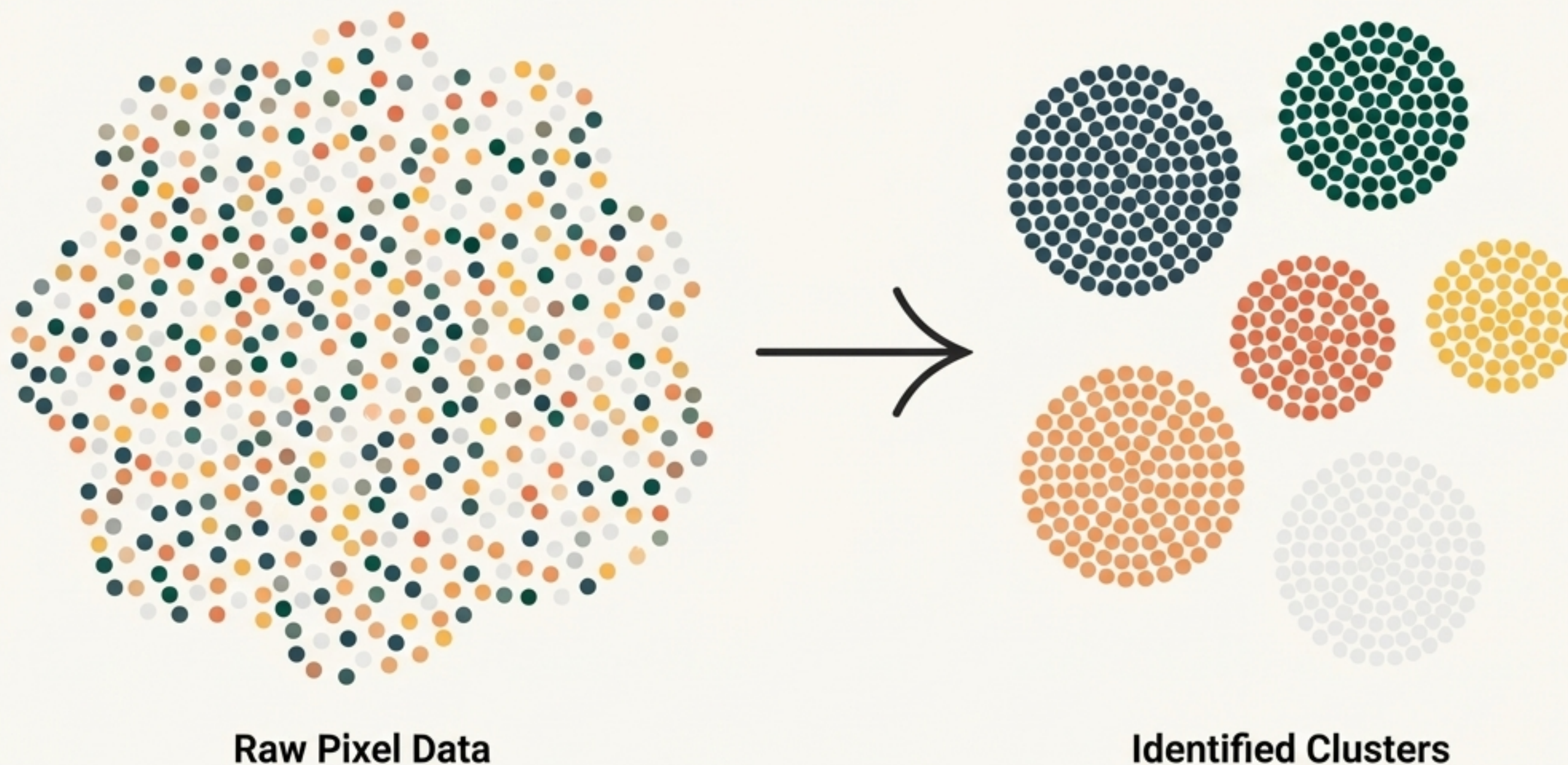
The Analyst's Core Challenge

An aerial satellite image of a landscape featuring a large body of water, forested areas, and some urban development. The image is overlaid with a light gray grid, suggesting a spatial analysis or mapping context.

You have a vast, detailed satellite image. How do you map its distinct distinct features—water, forests, urban areas—with no pre-existing labels or training data?

The Approach: Finding Patterns with Unsupervised Classification

Unsupervised classification, or **clustering**, is used when you do not have pre-labelled training data. The algorithm identifies and groups similar pixels together, determining distinct land cover classes on its own.

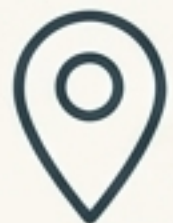


In Earth Engine

This process is managed by a `Clusterer` object. It functions like a supervised classifier but requires no manual class attributes to be provided.
Source Serif Pro Regular

Step 1: Creating a Clean Composite Image

Before we can classify the landscape, we must first isolate our study area and distil thousands of satellite images into a single, perfect snapshot. This involves three key actions:



Select by Location: Narrowing down to our specific Region of Interest (ROI).



Select by Time: Defining a clear date range (e.g., 2018-2020).



Ensure Quality: Filtering for images with minimal cloud cover (<10%).



Step 1: The Code

```
var selection = L8.filterBounds(ROI)
    .filterDate("2018", "2020")
    .filterMetadata("CLOUD_COVER", "less_than", 10)
    .median();
```

Selects the Landsat 8 image collection, then filters it to only images that cover our ROI.

Further narrows the collection to a specific time window and low cloud cover.

Crucially, this function reduces the entire filtered collection into a single composite image. The next step, `.sample()`, requires a single image to draw from, not a collection.

Step 2: Gathering Representative Training Samples

Instead of analysing the entire image at once—which is computationally intensive—we first gather a small, random set of training points. The algorithm will learn the landscape's patterns from the spectral values of these points.



Efficiency:

Using a small sample (e.g., 150 pixels) ensures the process runs quickly.



Reproducibility: We ensure the 'random' selection is the same every time, making our results consistent and repeatable.



Step 2: The Code

```
var trainingPoints = selection.sample({  
    region: ROI,  
    scale: 30,  
    numPixels: 150,  
    seed: 42  
});
```

Defines the number of random points to select. A small number is used here for speed and computational efficiency.

This ensures that the exact same 'random' points are chosen every time the script is run, making the experiment reproducible.

Step 3: Training the Clusterer with WEKA K-Means

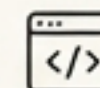
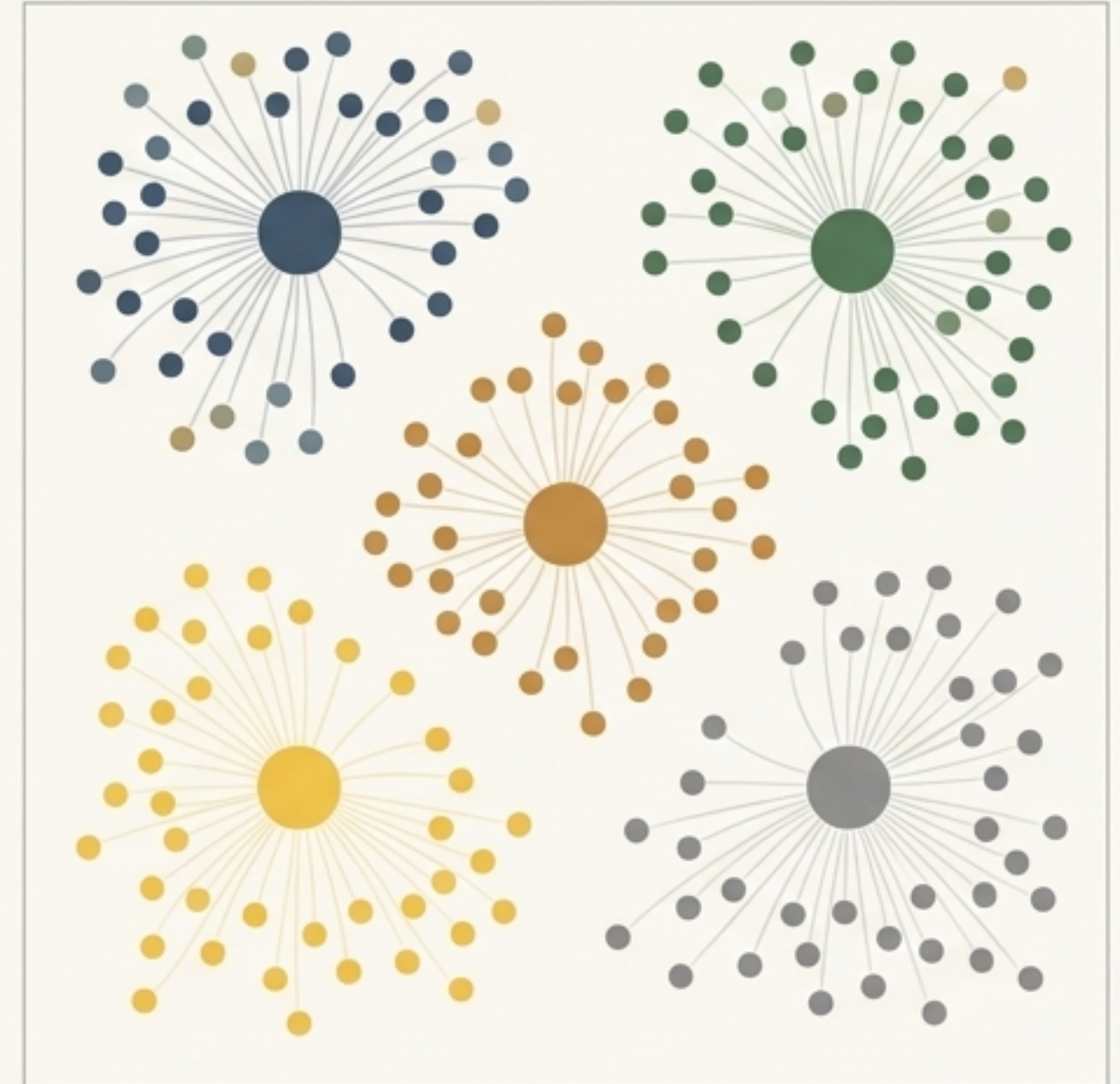
Now we train our `Clusterer` object using the sample points. We use the K-Means algorithm, a powerful tool that automatically groups the data into a specified number of clusters.

🕒 How it Works

The algorithm analyses the spectral values of our 150 sample points and iteratively adjusts 'cluster centres' until it finds the best mathematical groupings.

👤 The Human Instruction

Our only input is to tell the algorithm *how many* groups to find.



Technology Note: Earth Engine's clustering algorithms are powered by **WEKA**, a well-established data mining toolkit from the University of Waikato, New Zealand.

Step 3: The Code

```
var clusterer = ee.Clusterer.wekaKMeans(5)  
  .train(trainingPoints);
```

This initialises the K-Means clusterer from the WEKA toolkit. The number (5) is the crucial instruction: we are telling the algorithm to find exactly **5** distinct categories in our data.

This executes the training process, feeding our 150 sample points to the algorithm to build the classification model.

Step 4: Applying the Model to the Entire Landscape

Objective

The `clusterer`` has now learned the patterns from our 150-point sample. The final step is to apply these learned rules to **every single pixel** in our composite image.

The Visualisation Challenge

The algorithm outputs numerical clusters (0, 1, 2, 3, 4). These are just numbers; they don't have inherent colours. We need a final step to assign a unique, random colour to each cluster so we can see the result on a map.



Step 4: The Code

```
// Apply the clusterer to the entire image.  
var clusteredImage = selection.cluster(clusterer);
```

This is the command that applies the trained rules to every pixel in the `selection` image, generating the final classified map.

```
// Visualise the result.  
Map.addLayer(clusteredImage.randomVisualizer(), {}, 'Unsupervised Clusters');
```

A utility function that assigns a unique, random colour to each numerical cluster (0, 1, 2, 3, 4), allowing us to see the spatial distribution of the classes.

The Result: From Composite to Classification



Input: Median Composite



Output: Classified Image

The workflow successfully transformed a raw satellite image into a thematic map with five distinct spectral classes, ready for interpretation.

The Final and Most Critical Step: Human Interpretation

The algorithm only provides numbered groups; it does not know what they represent. The final step is for you, the expert, to provide meaning.

Cluster 3 (Deep Green)
appears to be dense forest.

Cluster 0 (Deep Slate Blue)
consistently corresponds
to water bodies.



Cluster 1 (Grey) is likely
urban or developed land.

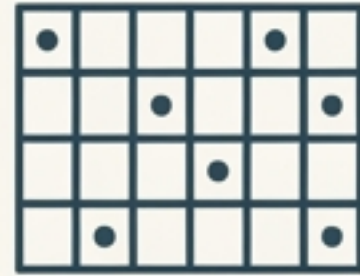
The output of unsupervised classification is not the final answer; it is a powerful starting point for expert-driven analysis and insight.

Workflow Summary



Filter & Composite

Isolate a region and time, then create a single, cloud-free image using `.median()`.



Sample

Gather a small, reproducible set of training points using `.sample()` for efficiency.



Train Clusterer

Use `.wekaKMeans()` to train a model on the samples, defining the number of classes to find.



Classify & Visualise

Apply the trained model to the entire image with `.cluster()` and display the result.